

What is Monitoring and Instrumentation?

What is a Metric?

A numeric representation of a measurement, such as CPU usage.

What is Monitoring?

The process of collecting, analyzing, and visualizing metrics

What is Kubernetes Monitoring?

The collection and analysis of metrics to ensure the performance, health, and reliability of a Kubernetes cluster



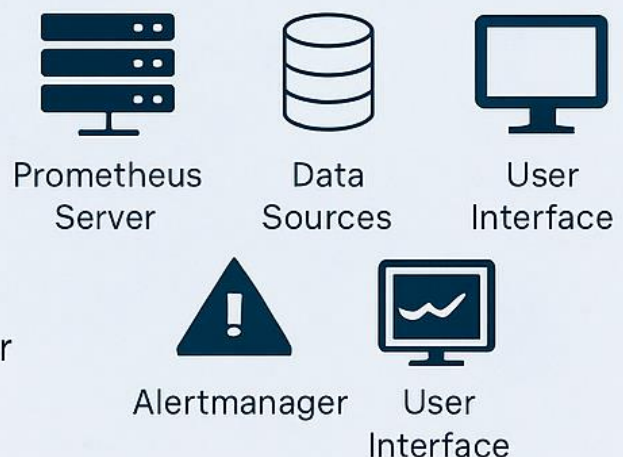
What is Prometheus?

An open-source systems monitoring and alerting toolkit

What is Kubernetes Monitoring?

the collection and analysis of metrics to ensure the performance, health, and reliability of a Kubernetes cluster

Prometheus Architecture Components:



What is Grafana?

To visualize, analyze, and connect data from various sources

Why Use Grafana?

to visualize, analyze, and connect data from various sources

What is a Metric?

A **metric** is a **quantitative measurement** used to track and understand the performance, usage, or behavior of a system.

Example of a Metric:

CPU usage of a pod: pod_cpu_usage_seconds_total

Number of HTTP requests: http_requests_total

Memory usage: container_memory_usage_bytes

What is Monitoring?

Monitoring is the **process of collecting, analyzing, and visualizing metrics** over time to understand the health and performance of systems and applications.

Example of Monitoring:

Using a dashboard to see **CPU and memory usage** of all Kubernetes nodes.

Setting up an alert if a pod restarts more than 3 times in 5 minutes.

What is Kubernetes Monitoring?

Kubernetes monitoring is the practice of **tracking the state and performance of Kubernetes clusters, nodes, pods, and containers**.

Why is Kubernetes Monitoring Important?

Track Node/Pod Health:

Monitor the status of pods, nodes, containers, and services in real-time.

Debugging and Alerting:

Detect failures (crashed pods, high memory usage) and trigger alerts automatically.

Performance Optimization:

Optimize cluster performance by understanding workload behavior and bottlenecks.

Auto-Scaling Support:

Horizontal Pod Autoscaler uses metrics (like CPU) to auto-scale pods based on demand.

Security and Compliance:

Monitor unusual behavior, unauthorized access, or resource over-utilization.

Resource Planning:

Forecast resource requirements and reduce costs based on usage patterns.

What is Prometheus?

Prometheus is an **open-source monitoring and alerting toolkit** designed specifically for cloud-native environments like Kubernetes.

Prometheus Architecture Components:

Prometheus Server:

Prometheus server is the core of the monitoring system. It is responsible for scraping metrics from various configured targets, storing them in its time-series database (TSDB), and serving queries through its HTTP API.

Components:

Retrieval: This module handles the scraping of metrics from endpoints, which are discovered either through static configurations or dynamic service discovery methods.

TSDb (Time Series Database): The data scraped from targets is stored in the TSDb, which is designed to handle high volumes of time-series data efficiently.

HTTP Server: This provides an API for querying data using PromQL, retrieving metadata, and interacting with other components of the Prometheus ecosystem.

Storage: The scraped data is stored on local disk (HDD/SSD) in a format optimized for time-series data.

Service Discovery:

Service discovery automatically identifies and manages the list of scrape targets (i.e., services or applications) that Prometheus monitors.

This is crucial in dynamic environments like Kubernetes where services are constantly being created and destroyed.

Components:

Kubernetes: In Kubernetes environments, Prometheus can automatically discover services, pods, and nodes using Kubernetes API, ensuring it monitors the most up-to-date list of targets.

File SD (Service Discovery): Prometheus can also read static target configurations from files, allowing for flexibility in environments where dynamic service discovery is not used.

Pushgateway:

The Pushgateway is used to expose metrics from short-lived jobs or applications that cannot be scraped directly by Prometheus.

These jobs push their metrics to the Pushgateway, which then makes them available for Prometheus to scrape (pull).

Use Case:

It's particularly useful for batch jobs or tasks that have a limited lifespan and would otherwise not have their metrics collected.

Alertmanager:

The Alertmanager is responsible for managing alerts generated by the Prometheus server.

It takes care of deduplicating, grouping, and routing alerts to the appropriate notification channels such as PagerDuty, email, or Slack.

Exporters:

Exporters are small applications that collect metrics from various third-party systems and expose them in a format Prometheus can scrape. They are essential for monitoring systems that do not natively support Prometheus.

Types of Exporters:

Common exporters include the [Node Exporter](#) (for hardware metrics), the [MySQL Exporter](#) (for database metrics), and various other application-specific exporters.

Prometheus Web UI:

The Prometheus Web UI allows users to explore the collected metrics data, run ad-hoc PromQL queries, and visualize the results directly within Prometheus.

Grafana:

Grafana is a powerful dashboard and visualization tool that integrates with Prometheus to provide rich, customizable visualizations of the metrics data.

API Clients:

API clients interact with Prometheus through its HTTP API to fetch data, query metrics, and integrate Prometheus with other systems or custom applications.

What is Grafana?

Grafana is an **open-source visualization and analytics platform**. It connects to Prometheus (and other data sources) to create **custom dashboards** for monitoring.

Why Use Grafana?

Rich visualizations: Graphs, heatmaps, tables, gauges, etc.

Flexible dashboard creation with filters, variables, and alerts.

Supports multiple data sources: Prometheus, Loki, InfluxDB, Elasticsearch, MySQL, etc.

When is Grafana Used?

When you want to **create real-time dashboards** for Kubernetes clusters.

For **centralized monitoring** of multiple systems.

To set up **alerts and reports** using visual rules and thresholds.

Feature	Prometheus	Grafana
Type	Monitoring and alerting system	Visualization and analytics tool
Storage	Local TSDB	Doesn't store metrics (only visualizes)
Language	PromQL	Custom UI expressions
Data Collection	Pull-based scraping	Connects to Prometheus to display
Alerting	Yes (via Alertmanager)	Yes (can also integrate with alerts)
Use Case	Metric collection, alerts	Dashboard visualization